

1. Each signal also has a default action associated with it. It could be either of the four mentioned below:

- **Exit:** Makes the process exit.
- **Core:** Forces the process to exit and create a core dump file.
- **Stop:** Stops/suspends the process.
- **Ignore:** No action taken.

The framework is also flexible enough, so you can change the default disposition of a signal to one of your choice by overriding the default signal handler.

The framework also allows a given process to block some signals so that they don't get delivered to the process at all.

One restriction imposed here by the system is that the process cannot change the default disposition for **SIGKILL** and **SIGSTOP**.

Practical applications

This section talks about the various standard applications based on 'signals' in a typical Linux system.

IPC 302: Yes! This is the basic one, or the *kill* command. It sends the **SIGTERM** to a process specified in the argument, and the process terminates. You can also send **SIGKILL** to kill a process through the same command if the process ignores **SIGTERM**.

Ctrl+C: When you press **Ctrl+C** on the keyboard, the process running on the foreground on the given terminal receives **SIGINT**. The process also terminates by default when it receives **SIGINT**.

Old buddy GDB: Yes, the working of GDB is totally based on the signals **SIGSTOP** and **SIGCONT**. When the process being debugged reaches a breakpoint, GDB sends **SIGSTOP** to a given process and its execution halts. Now, after looking at the available information, when a user makes the process run, GDB sends out **SIGCONT** which clears **SIGSTOP** and lets the process go ahead.

Alarms: When an application wants to run the timers, they typically make use of APIs like *setitimer()* specifying the time out value in the arguments. When the timer expires, the process gets **SIGALARM**.

Child care: In the Linux system, process creations are done through the *fork()* system call and the processes have a parent-child relationship. Now

parents need to be informed when a child changes its state so that they can take appropriate action, such as doing a cleanup, spawning one more child if one gets killed, etc. This functionality is achieved through **SIGCHLD**.

Broken pipe: In Linux, processes pass the data to other processes through pipes/fifos/sockets, etc. When a process attempts to write to a broken pipe, the process receives **SIGPIPE** indicating the same.

Check your Memory/Math/Instruction set: When a process attempts to access an invalid memory address, it receives a **SIGSEGV** from the system. Similarly, when a program attempts to execute invalid floating point computation, it receives **SIGFPE** for it. Also, if a process attempts to execute illegal instruction, it gets **SIGILL** indicating the same.

By default, these signals also result in programs to crash with a core dump.

Something left for programmers: There are two signals **SIGUSR1** and **SIGUSR2**, which are left for the programmers, and their meanings need to be set by them.

API support for signal handling

There are two main APIs available for programmers to change the default disposition of the signals. The first is *signal()*, which looks like what's shown below:

```
void (* signal (int sig, void (*func)(int)))(int);
```

The equivalent *typedef* version for the same, which is easier to read, is as follows:

```
typedef void (*sig_t) (int);
```

```
sig_t signal(int sig, sig_t func);
```

The function is very simple to use. You only need to specify the signal number and call-back function that needs to be registered. But the API is getting deprecated, and a more robust and elaborate API called *sigaction()* is available:

```
int sigaction(int sig, const struct sigaction *act, struct sigaction *oact);
```

